

Business Observability, RUM & Synthetics

Connect telemetry to business outcomes, then monitor the real and simulated user journey end to end — business KPI instrumentation, Real User Monitoring with Core Web Vitals, and Synthetic testing, with diagrams, hands-on labs, solutions, and review questions.

Technical vs Business Metrics

KPI Instrumentation

E-commerce / Banking / SaaS KPIs

Real User Monitoring

Core Web Vitals: LCP · INP · CLS · FCP

Session Replay & Segmentation

Synthetic API & Browser Tests

Prepared for: Vivek Arora

Format: Certification preparation • Full guide • Day 03

Modules: 07–09 • Labs: 6 • Review questions: 24

Version 1.0 — August 2026

Table of Contents

Day 03 lifts observability from the backend to the business and the browser. You connect telemetry to outcomes (Module 07), then monitor the journey as real users live it (Module 08) and as scheduled tests simulate it (Module 09).

Recap & how Day 03 fits	3
Module 07 — Business Observability	4
7.1 Technical vs business metrics	4
7.2 KPI instrumentation strategy	5
7.3 Domain KPIs: e-commerce, banking, SaaS	6
7.4 Hands-on labs, use cases & review questions	8
Module 08 — Real User Monitoring (RUM)	11
8.1 The RUM data model & experience monitoring	11
8.2 Core Web Vitals: LCP, FCP, CLS, INP	12
8.3 Session Replay & user segmentation	13
8.4 Hands-on lab, use cases & review questions	14
Module 09 — Synthetic Monitoring	15
9.1 Simulating the user journey: API & browser tests	15
9.2 RUM vs Synthetics — using both	16
9.3 Hands-on lab, use cases & review questions	17
Appendix A — Answer keys	18
Appendix B — KPI & Core Web Vitals cheat sheet	21
Appendix C — Sources & further reading	23

Recap & how Day 03 fits

A one-page bridge from Days 01–02.

Day 01 covered *how trace data is produced and collected* (spans, the agent pipeline, sampling, the service map). Day 02 covered *analysing* that data — the Trace Explorer, latency percentiles, slow-transaction and error analysis — and expressing health as the four KPI families (availability, performance, reliability, user experience).

Day 03 pushes outward in two directions. **Module 07 (Business Observability)** connects that backend telemetry to the outcomes the business cares about — conversion, revenue, transaction success — and shows how to instrument business KPIs as first-class metrics. **Module 08 (RUM)** moves to the browser and the real user, capturing the front-end experience and Google's Core Web Vitals. **Module 09 (Synthetics)** adds the *simulated* journey — scheduled tests that watch critical paths 24/7, even when no real user is online. Together, RUM and Synthetics give you end-to-end digital-experience coverage: what users actually feel, and what your key journeys do around the clock.

The through-line

Day 03's theme is *relevance*: telemetry only matters if it maps to something a stakeholder cares about. A p95 is interesting; "this p95 costs \$4k/minute in abandoned carts, and our checkout Synthetic just went red in Frankfurt" is *actionable*. Business observability, RUM, and Synthetics are how you make that leap.

Scope note & live defaults

Your Day 03 outline lists two modules (07, 08) but frames the day as "3 modules · 3 hands-on sessions" and the "real *and simulated* user journey." Module 09 (Synthetic Monitoring) completes that set and is included here. Core Web Vitals thresholds and RUM/Synthetics defaults evolve; confirm against the docs in Appendix C, and capture live UI screenshots where placeholders appear.

Business Observability

Make the business itself observable: distinguish technical from business metrics, instrument KPIs as first-class signals, and track the metrics that matter in e-commerce, banking, and SaaS.

7.1 Technical vs business metrics

Business observability is the practice of measuring business outcomes with the same rigour — and often the same telemetry — as technical health, and correlating the two. It answers the question executives actually ask: "so what does this latency cost us?"

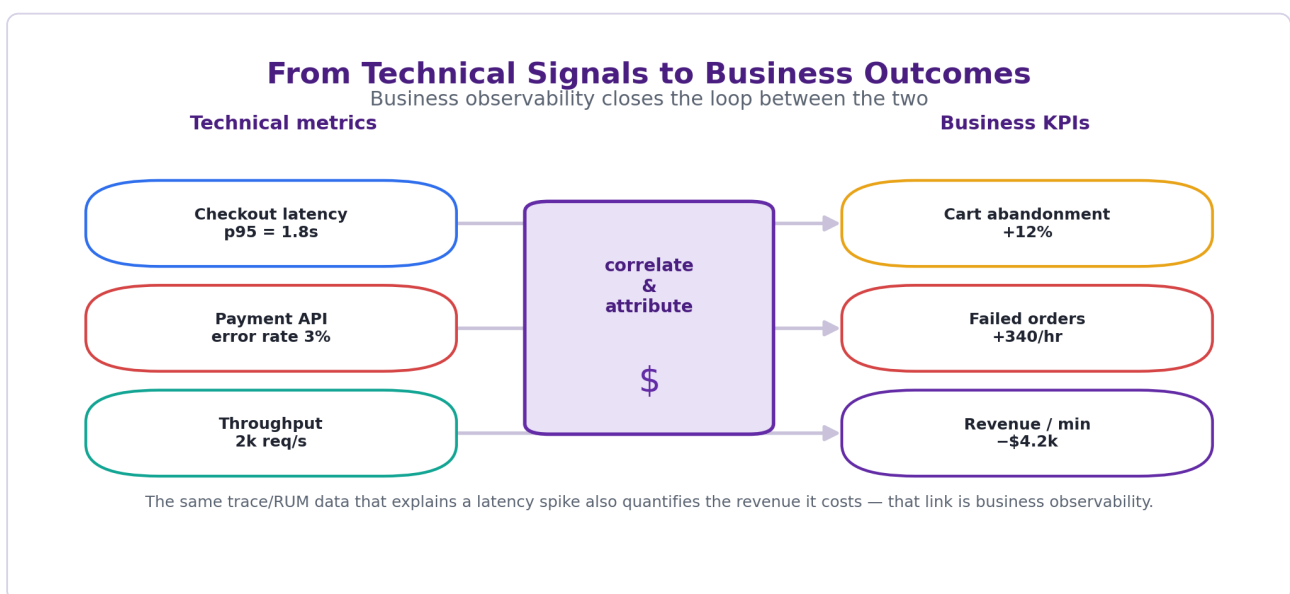


Figure 7.1 — The same trace/RUM data that explains a technical problem can quantify the business impact it causes.

Definition — Technical vs business metric

A **technical metric** measures how the *system* behaves — latency, error rate, throughput, CPU. A **business metric (KPI)** measures a *business outcome* — orders placed, revenue per minute, sign-ups, cart abandonment. The former is a means; the latter is the end. Business observability links them so a change in one is visible against the other on the same timeline.

Dimension	Technical metric	Business metric (KPI)
Measures	System behaviour	Business outcome
Audience	Engineers, SRE	Product, finance, execs
Examples	p95 latency, error rate, RPS	Conversion %, revenue/min, churn
Unit	ms, %, req/s	\$, orders, users
Source	APM, infra, logs	Custom/span/RUM/log-based metrics

Exam focus

Be able to classify a metric as technical or business and explain the value of correlating them: it converts an engineering event ("p95 rose") into a business decision ("we lost \$X; prioritise the fix"). The key phrase is *attribution* — tying a technical cause to a business effect on one dashboard.

7.2 KPI instrumentation strategy

A business KPI is only useful if it becomes a real, queryable metric. There are four common ways to get one into Datadog; choose by where the signal already lives.

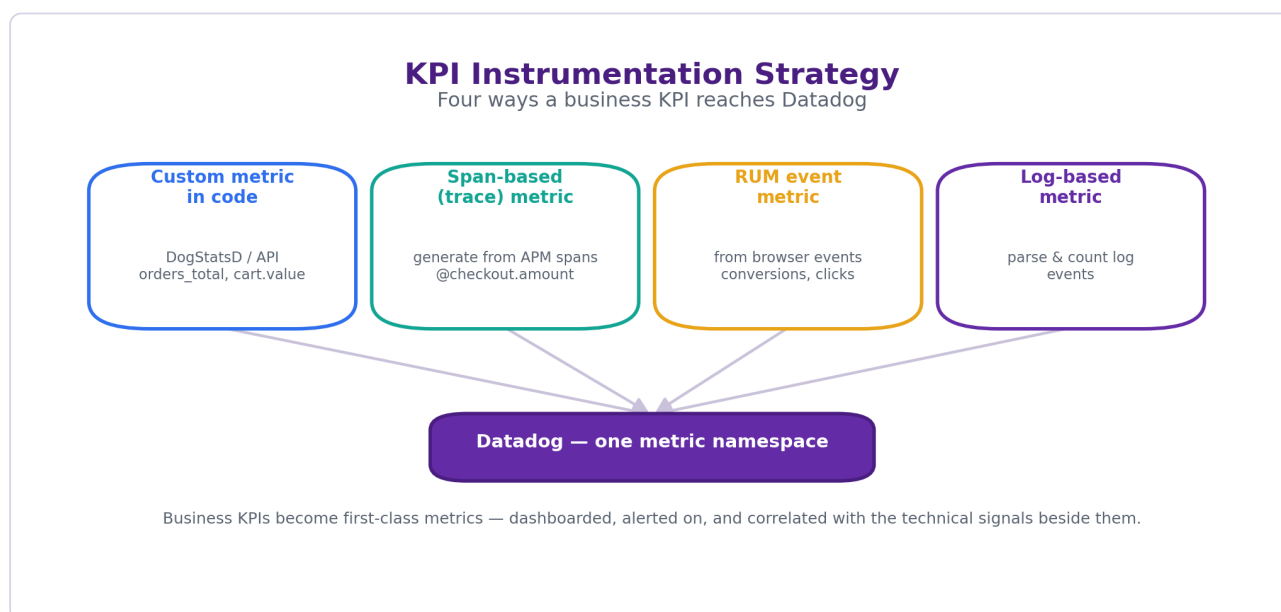


Figure 7.2 — Four routes to a business KPI: custom code metric, span-based metric, RUM-event metric, and log-based metric — all landing in one namespace.

Route	How	Best when
Custom metric in code	Emit via DogStatsD or the API (<code>statsd.increment('orders.placed')</code> , <code>gauge('cart.value')</code>)	The event is known in your code and you want full control.
Span-based (trace) metric	Generate a metric from APM spans/tags (e.g. sum of <code>@checkout.amount</code>)	The data is already on your traces — no new code.
RUM-event metric	Derive from browser events (conversions, clicks, views)	The KPI is a front-end/ user action.
Log-based metric	Parse and count matching log events	The signal only exists in logs (e.g. a legacy system).

```
# Custom business metrics via DogStatsD
statsd.increment('orders.placed', tags=['tier:premium', 'region:eu'])
statsd.gauge('cart.value_usd', 142.50, tags=['currency:usd'])
statsd.increment('checkout.completed') # numerator for conversion
```

Tip — Tag KPIs the way the business slices them

Attach the dimensions stakeholders actually pivot on — `region` , `plan` , `customer_tier` , `payment_method` , `device` . A revenue metric you can't break down by region or plan answers far fewer questions. Keep an eye on cardinality (Day 01) so custom-metric costs stay sane.

Watch out — instrument the outcome, not a proxy

"Add to cart" is not "purchase"; "page load" is not "conversion." Instrument the actual business event you care about. Proxies drift from reality and quietly mislead — a rising add-to-cart with flat revenue is a classic false comfort.

7.3 Domain KPIs: e-commerce, banking, SaaS

Different businesses live and die by different numbers. The conversion funnel is the backbone for transactional businesses; each stage drop is a KPI.

The Conversion Funnel & E-commerce KPIs

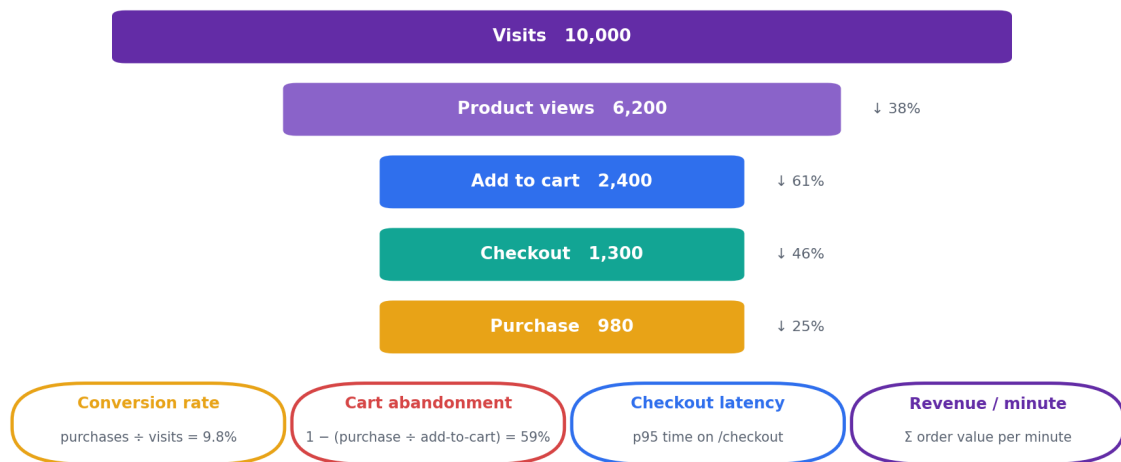


Figure 7.3 — The conversion funnel with the headline e-commerce KPIs derived from it.

E-commerce KPIs

KPI	Definition
Conversion rate	$\text{purchases} \div \text{visits (or sessions)} \times 100$
Cart abandonment rate	$1 - (\text{purchases} \div \text{add-to-carts}) \times 100$
Checkout latency	Response time (p95/p99) of the checkout flow — directly correlates with abandonment
Revenue per minute	$\Sigma \text{ order value per minute}$ — the real-time pulse of the business
Average order value (AOV)	$\text{total revenue} \div \text{number of orders}$

Banking / fintech KPIs

KPI	Definition
Transaction success rate	$\text{successful transactions} \div \text{total attempted} \times 100$
Payment processing time	Latency of a payment/transfer end-to-end (percentiles)
Failed transaction %	$\text{failed} \div \text{total}$ — split by reason (declines, timeouts, fraud holds)
Settlement / reconciliation lag	Time from authorisation to settlement

SaaS KPIs

KPI	Definition
Active users (DAU / MAU)	Distinct users active per day/month; DAU÷MAU ≈ stickiness
Feature adoption	Share of users/accounts using a given feature
Subscription conversion	trials converting to paid ÷ total trials
Churn rate	customers lost ÷ customers at start of period

Definition — Leading vs lagging KPIs

A **lagging** KPI reports an outcome after the fact (monthly churn, revenue). A **leading** KPI predicts it (checkout latency, failed-transaction %, feature adoption). Observability shines at *leading* KPIs: you can watch checkout latency climb in real time and act *before* the lagging revenue number drops.

Exam focus

Know at least two headline KPIs per domain and how each is computed, and be able to name the technical metric that most directly influences it (e.g. checkout latency → cart abandonment; payment processing time → transaction success rate). The recurring pattern: a performance/error metric is the *leading indicator* of a revenue/success KPI.

7.4 Hands-on labs, use cases & review questions

Use case — Attributing revenue to a latency regression

A deploy raises checkout p95 from 400 ms to 1.6 s. Engineering sees the latency graph; leadership doesn't care until it's translated. The team overlays `revenue_per_minute` and `cart_abandonment_rate` (both instrumented as metrics) on the same dashboard as checkout p95. The correlation is stark: abandonment jumps 11 points and revenue/min drops ~\$3.8k the moment latency rises. Now the fix is prioritised as a revenue incident, not a "nice-to-have perf tweak." *Lesson:* business observability turned a technical graph into a funded decision.

Lab 7.1 — Create business metrics

Goal: choose the right instrumentation route and define KPIs correctly.

You must expose three KPIs for a retail app: (a) conversion rate, (b) revenue per minute, (c) cart abandonment rate. For each, state the raw signal(s) you need, the best instrumentation route (custom / span-based / RUM / log-based), the tags you'd attach, and the exact formula.

Solution

(a) Conversion rate = purchases ÷ visits. Signals: a `checkout.completed` count and a `session / visit` count. Route: **RUM-event metric** for visits (front-end) and either a custom `orders.placed` or a **span-based** metric on the checkout service for purchases. Tags: `region`, `device`, `plan`. Compute as a ratio in the dashboard/monitor. **(b) Revenue per minute** = Σ order value/min. Signal: `order amount`. Route: **custom metric** `statsd.gauge('order.value_usd', amount)` or **span-based** sum of `@checkout.amount`; graph as a per-minute sum. Tags: `currency`, `region`, `payment_method`. **(c) Cart abandonment** = $1 - (\text{purchases} \div \text{add-to-carts})$. Signals: `cart.add` and `checkout.completed` counts (custom or RUM actions). Compute the ratio and subtract from 1. In all cases: instrument the *real* event, and tag for the slices the business pivots on.

Lab 7.2 — Build a business dashboard

Goal: lay out a dashboard that a business stakeholder and an engineer can both read.

Design a single "Checkout Health & Revenue" dashboard. Specify the panels, how you pair technical and business signals, and one correlation panel that would let an on-call engineer prove (or rule out) that a technical regression is hurting revenue.

Solution

Top row (business): revenue/min (timeseries), conversion rate (query value with threshold), cart abandonment (query value). **Second row (technical):** checkout p95/p99 latency and error rate/success rate. **Correlation panel:** a single graph overlaying *checkout p95* and *revenue/min* (dual axis) — if revenue dips exactly when latency spikes, the link is visible at a glance. Add **template variables** for `region` and `version` so you can isolate a bad deploy or a regional CDN issue. Layout principle from Day 02: put the outcome (revenue, conversion) top-left where it's read first, with the technical causes directly beneath, so the eye travels effect → cause. Attach monitors to conversion rate *and* checkout error rate.

Review questions — Module 07

Q1. Define business observability in one sentence.

Q2. Classify each as technical or business: p99 latency, revenue per minute, RPS, cart abandonment rate, error rate, feature adoption.

- Q3.** Name the four routes to instrument a KPI and when each is the best choice.
- Q4.** Why should business KPIs be tagged with dimensions like region and plan?
- Q5.** Give the formulas for conversion rate and cart abandonment rate.
- Q6.** State two banking KPIs and two SaaS KPIs with their definitions.
- Q7.** Distinguish a leading KPI from a lagging KPI, with one example of each, and say why observability favours leading KPIs.
- Q8.** Why is "add to cart" a poor proxy for "purchase"?

Answers in Appendix A.

Real User Monitoring (RUM)

See the application through the eyes of the people using it: the RUM data model, front-end performance and Core Web Vitals, and session-level insight through replay and segmentation.

8.1 The RUM data model & experience monitoring

Real User Monitoring instruments the front end (browser or mobile app) to capture what real users actually experience — page loads, interactions, errors, and resource timings — from their real devices and networks. Where APM traces the backend, RUM covers the "last mile" the user sees.

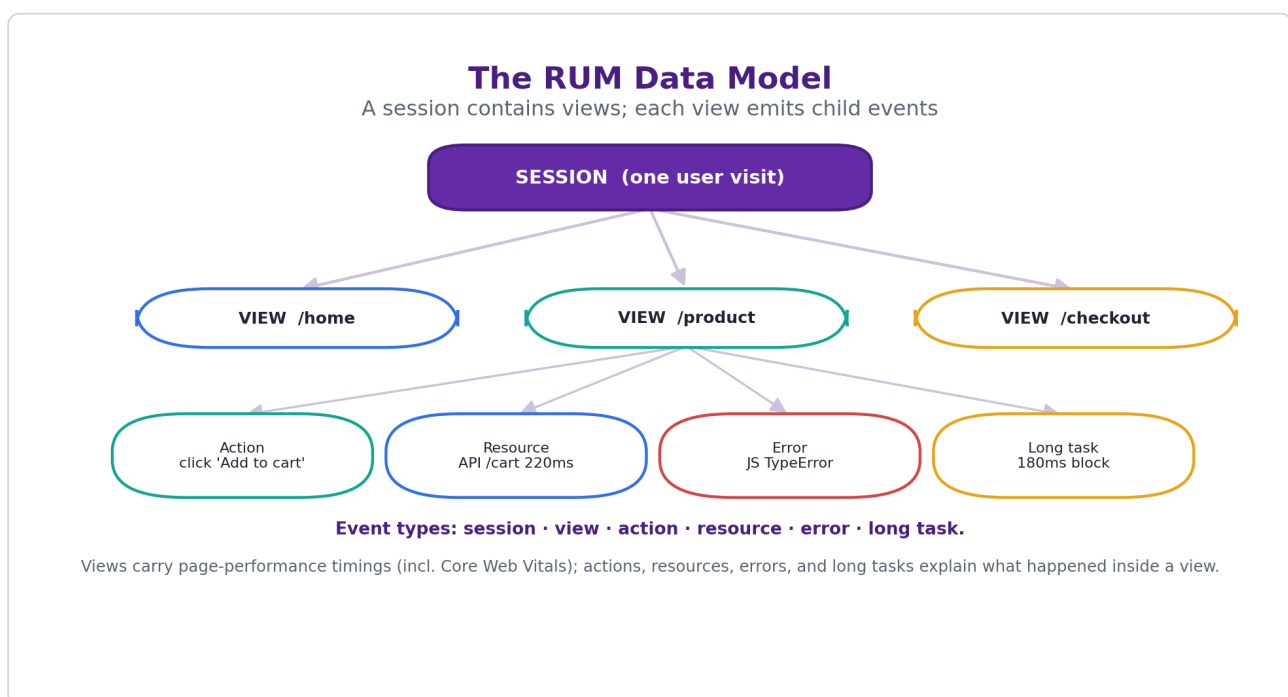


Figure 8.1 — RUM's hierarchy: a session contains views; each view emits action, resource, error, and long-task events.

Definition — RUM event types

RUM emits six event types. A **session** is one user's visit (a series of views within a time window). A **view** is a page load or screen (it carries load timings and Core Web Vitals). Inside a view: **action** events (clicks, taps), **resource** events (network requests — XHR, fetch, images, CSS/JS), **error** events (JS exceptions, failed requests), and **long task** events (main-thread blocks > 50 ms that hurt responsiveness).

User-journey monitoring uses these events to follow a real path — home → product → checkout — measuring where users drop off and what went wrong (a JS error, a slow API resource, a layout shift). Because RUM resources can be linked to backend APM traces, you can pivot from a slow front-end request straight into the server-side flame graph — true end-to-end visibility.

RUM concept	What it captures
Frontend performance	Load timings, Core Web Vitals, resource waterfalls per view
Browser analytics	Aggregate views by page, browser, OS, device, country, connection
User journey	Ordered sequence of views/actions across a session
RUM → APM link	Correlates a front-end resource with its backend trace

Exam focus

Memorise the six RUM event types and their hierarchy (session > view > action/resource/error/long task). Know that *views* are where Core Web Vitals live, and that RUM's value over synthetic/backend-only monitoring is that it reflects *real* device, network, and geographic diversity.

8.2 Core Web Vitals: LCP, FCP, CLS, INP

Core Web Vitals are Google's standardised front-end UX metrics, each capturing a different facet of experience. Datadog RUM measures them from real users and recommends judging them at the **75th percentile**.

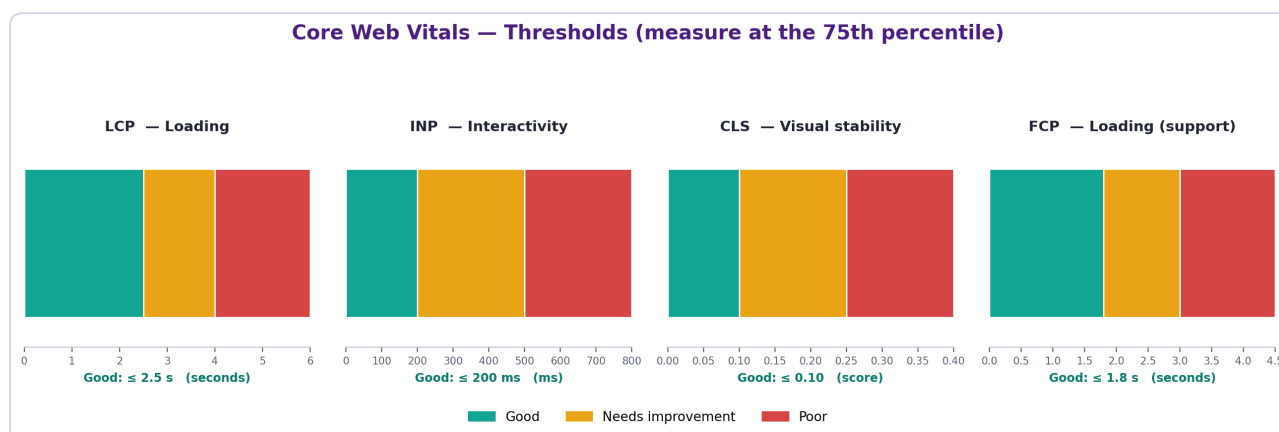


Figure 8.2 — The Core Web Vitals and their good / needs-improvement / poor thresholds.

Metric	Measures	Good	Poor
LCP — Largest Contentful Paint	Loading: when the largest element renders	≤ 2.5 s	> 4.0 s
INP — Interaction to Next Paint	Interactivity: responsiveness to input	≤ 200 ms	> 500 ms
CLS — Cumulative Layout Shift	Visual stability: unexpected movement	≤ 0.10	> 0.25
FCP — First Contentful Paint	Loading (support): first content painted	≤ 1.8 s	> 3.0 s

Definition — INP replaced FID

INP (Interaction to Next Paint) became a Core Web Vital in March 2024, replacing **FID (First Input Delay)**. FID measured only the delay of the *first* interaction; INP is stricter — it reflects the worst interaction latency across the whole visit, so it better represents ongoing responsiveness. If a question mentions FID, know it is deprecated in favour of INP.

Each metric points to a different class of fix. **LCP** is improved by faster servers/CDN, optimised images, and removing render-blocking resources. **INP** is improved by reducing main-thread work and long tasks (breaking up heavy JS). **CLS** is improved by reserving space for images/ads and avoiding inserting content above existing content. **FCP** is a supporting loading metric that helps diagnose LCP problems.

Tip — Always read Core Web Vitals at p75, segmented

A p75 target means "75% of real visits meet it." Averages hide the slow quarter (Day 02's lesson, applied to the front end). Then *segment* — CWV often differ sharply by device (mobile vs desktop), country, and connection type; a "good" global LCP can hide a "poor" mobile-on-3G LCP.

8.3 Session Replay & user segmentation

Beyond metrics, RUM offers qualitative and cohort-level insight.

Definition — Session Replay

Session Replay reconstructs a real user's session as a privacy-conscious, video-like playback of the DOM — every view, click, scroll, and error — so you can *watch* exactly what a user did and saw when something went wrong. It turns "an error occurred" into "here's the rage-click on the disabled button that caused it." Sensitive content can be masked to protect privacy.

Definition — User segmentation

Segmentation slices RUM data by user and context attributes — device, browser, OS, country, connection type, app version, or custom attributes like `plan` or `user.id`. It answers "who is affected?" — e.g. "the checkout error only hits Safari on iOS in the EU," turning a vague complaint into a targeted fix.

Exam focus

Session Replay = qualitative *what happened in this session*; segmentation = *which cohorts are affected*. Both complement the quantitative Core Web Vitals. Expect a question on using segmentation to localise an issue to a device/browser/region.

8.4 Hands-on lab, use cases & review questions

Use case — The invisible mobile regression

Overall LCP looks healthy at p75 = 2.3 s, but support reports "the site feels slow." Segmenting RUM by `device` reveals mobile p75 LCP = 4.6 s (poor) while desktop is 1.6 s — the desktop majority masked it. Filtering to the worst views and opening a Session Replay shows a giant hero image loading last on mobile. Reserving space and serving a responsive image drops mobile LCP to 2.1 s. *Lesson:* aggregate CWV hid a device-specific problem that segmentation + replay exposed.

Lab 8.1 — Diagnose a Core Web Vitals problem

Goal: map symptoms to the right Core Web Vital and fix.

For each user complaint, name the Core Web Vital most implicated and one likely fix: (a) "the page jumps around while loading and I click the wrong thing"; (b) "it takes forever for the main article/image to show up"; (c) "buttons feel laggy — there's a delay after I click"; (d) "the page looks blank for a moment before anything appears."

Solution

(a) **CLS** (visual stability) — reserve explicit width/height for images and ads, and avoid inserting content above existing content. (b) **LCP** (loading) — optimise/preload the hero image, use a CDN, and remove render-blocking JS/CSS. (c) **INP** (interactivity) — reduce main-thread long tasks, break up heavy JavaScript, defer non-critical work. (d) **FCP** (first paint) — cut server response time (TTFB) and render-blocking resources so first content paints sooner. Confirm each at **p75** and segment by device before and after.

Review questions — Module 08

Q9. List the six RUM event types and their hierarchy.

Q10. Which RUM event carries Core Web Vitals?

Q11. Name the four Core Web Vitals in this guide and what each measures.

Q12. Give the "good" thresholds for LCP, INP, and CLS.

Q13. Which metric replaced FID, and why is it considered stricter?

Q14. At what percentile does Datadog recommend evaluating Core Web Vitals, and why not the mean?

Q15. What is Session Replay and what class of problem does it best solve?

Q16. How does user segmentation help localise an issue? Give an example.

Q17. Name one fix each for a poor LCP, a poor INP, and a poor CLS.

Answers in Appendix A.

Synthetic Monitoring

Monitor the *simulated* user journey: scheduled API and browser tests that watch your critical paths from around the world, 24/7, whether or not a real user is online.

9.1 Simulating the user journey: API & browser tests

Synthetic Monitoring runs automated, scripted requests and interactions against your systems from chosen locations, on a schedule. Because the load is generated on demand, it gives a *consistent baseline* and can catch outages proactively — before real users hit them, and even during off-hours or before launch.

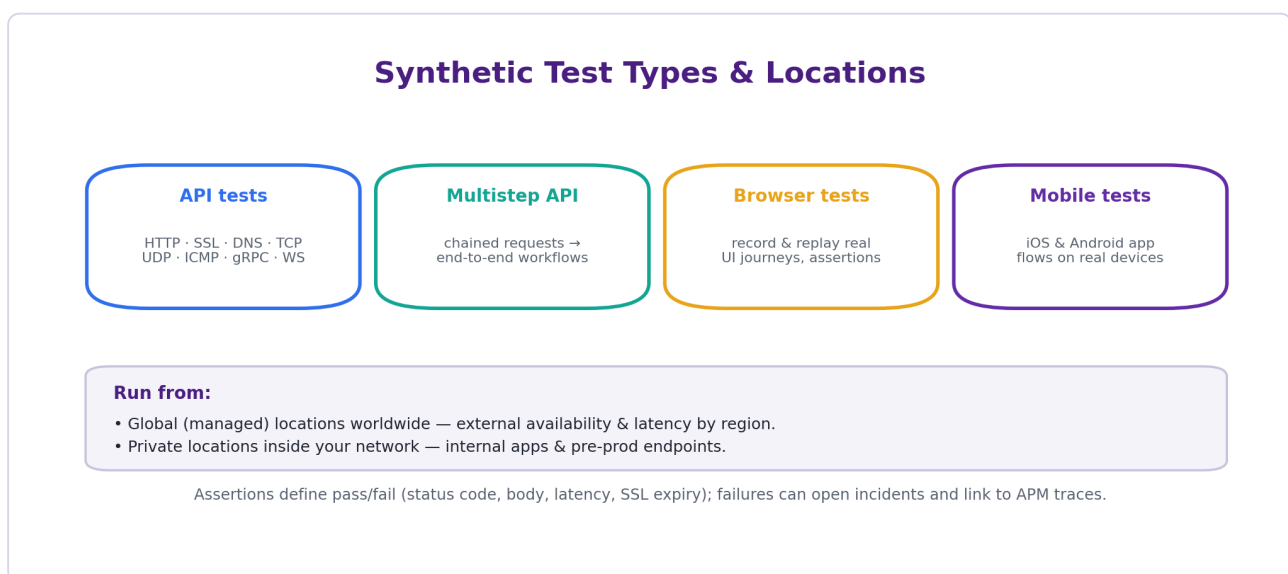


Figure 9.1 — Synthetic test types (API, multistep API, browser, mobile) and where they run (global vs private locations).

Test type	What it checks	Protocols / scope
API test	A single endpoint's availability, correctness, latency	HTTP, SSL, DNS, TCP, UDP, ICMP, gRPC, WebSocket
Multistep API test	A chained workflow across several requests (e.g. auth → create → verify)	Sequenced HTTP steps with extracted variables
Browser test	An end-to-end UI journey in a real browser (record & replay clicks, forms, assertions)	Chromium/Firefox, multiple viewports
Mobile test	App flows on real iOS/Android devices	Native mobile apps

Definition — Assertion, location, uptime SLA

An **assertion** defines pass/fail — status code = 200, response body contains a field, latency < 500 ms, SSL certificate not expiring within N days. **Global locations** are Datadog-managed points of presence worldwide (external availability & regional latency); **private locations** run inside your own network to test internal or pre-production endpoints. Synthetic results can compute uptime/SLA and open incidents on failure.

Definition — Multistep & variables

A **multistep API test** chains requests and passes data between them using **variables** — e.g. step 1 logs in and extracts a token; step 2 uses that token to place an order; step 3 verifies the order. This models a real transaction, not just a single endpoint ping, making it the API-level equivalent of a browser journey.

Failure handling & APM correlation. When a synthetic test fails, it can trigger alerts and incidents, and — because Datadog links the test to the backend it hit — you can jump from a failed test straight into the APM trace to see *why* it failed (a slow query, a 500 from a dependency). This closes the loop from "the checkout journey is down in Frankfurt" to the exact failing span.

Exam focus

Know the test families (API, multistep API, browser, mobile), that assertions define pass/fail, and the global-vs-private-location distinction. The headline value of Synthetics: *proactive, consistent, 24/7* coverage of critical paths, independent of real traffic.

9.2 RUM vs Synthetics — using both

RUM and Synthetics are complementary, not competing. RUM tells you what real users *actually* experienced (but only where you have traffic, and only after the fact); Synthetics tell you how a critical path performs *right now* from a controlled vantage point (but not with real-user diversity).

Real User Monitoring vs Synthetic Monitoring

Two complementary lenses on digital experience

REAL USER MONITORING

Actual users, real devices & networks

- ✓ True field experience
- ✓ Core Web Vitals from real users
- ✓ Segmentation, session replay
- ✗ Only where you already have traffic
- ✗ Can't test before release / off-hours

SYNTHETIC MONITORING

Simulated requests from chosen locations

- ✓ Proactive — before users hit it
- ✓ Consistent, scheduled baseline
- ✓ Covers critical paths 24/7
- ✓ Works pre-launch & off-hours
- ✗ Not real users / real diversity

Use both: Synthetics catch outages proactively on key journeys; RUM shows what real users actually experience.

Figure 9.2 — Real User Monitoring vs Synthetic Monitoring: two complementary lenses on digital experience.

	RUM (real)	Synthetics (simulated)
Data source	Actual users & devices	Scripted tests from chosen locations
Timing	Reactive — needs real traffic	Proactive — scheduled, pre-launch, off-hours
Coverage	Only pages users visit	Any critical path you script, 24/7
Strength	Real diversity, Core Web Vitals, replay	Consistent baseline, early outage detection, SLAs
Blind spot	Low-traffic / new pages	Not representative of real-user variety

Tip — A practical division of labour

Use **Synthetics** for uptime/SLA on your handful of business-critical journeys (login, search, checkout, payment) from key regions, running continuously. Use **RUM** to understand the real, diverse experience across everything users actually touch, to catch what your scripts didn't anticipate, and to measure Core Web Vitals in the field. The alert usually comes from Synthetics; the full story comes from RUM + APM.

9.3 Hands-on lab, use cases & review questions

Use case — Catching an outage before the users do

A certificate on the payments API silently approaches expiry over a weekend, when real traffic is low. A scheduled **SSL/API synthetic** with an assertion "certificate expires in > 7 days" fails on Saturday and pages the on-call — hours before Monday's traffic would have hit a wall of failed transactions. A separate **multistep browser test** for the checkout journey stays green, confirming the rest of the path is healthy. *Lesson:* Synthetics found, off-hours, a problem no real-user tool could surface because no users were there yet.

Lab 9.1 — Design end-to-end journey coverage

Goal: combine Synthetics and RUM into a coherent digital-experience monitoring plan.

For a retail site with login, search, and checkout, specify: which synthetic test type you'd use for each journey, the key assertions, which locations, and how you'd divide responsibilities between Synthetics and RUM. Then state what each layer would tell you during a real checkout outage in one region.

Solution

Synthetics: a *multistep browser test* for checkout (add to cart → enter details → pay → assert confirmation page + latency < 3 s), a *browser test* for search (query → assert results render), and a *multistep API test* for login (auth → token → profile fetch, assert 200 + body). Add an *SSL/API test* on the payments endpoint asserting cert validity. Run from several **global** locations covering your main markets, plus a **private** location for internal admin APIs; schedule every 1–5 minutes. **RUM** covers the whole site for real-user experience and Core Web Vitals, segmented by device/region. **During a regional checkout outage:** the checkout Synthetic from that region goes red first (proactive alert) and links to the failing APM trace; RUM confirms real users there are hitting errors and quantifies how many and on which devices; APM shows the backend root cause. Synthetics = detection, RUM = real impact, APM = cause.

Review questions — Module 09

Q18. What is Synthetic Monitoring, and what is its headline advantage over RUM?

Q19. Name the synthetic test types and what each is for.

Q20. What is an assertion? Give three examples.

Q21. Distinguish global from private locations.

Q22. What does a multistep API test add over a single API test?

Q23. Give one thing RUM can tell you that Synthetics cannot, and one thing Synthetics can tell you that RUM cannot.

Q24. During a regional outage, what role does each of Synthetics, RUM, and APM play?

Answers in Appendix A.

Appendix A — Answer keys

Module 07

#	Answer
Q1	Measuring business outcomes with the same rigour/telemetry as technical health and correlating the two, so a technical change is visible against its business impact.
Q2	Technical: p99 latency, RPS, error rate. Business: revenue per minute, cart abandonment rate, feature adoption.
Q3	Custom metric in code (event known in code, full control); span-based (data already on traces); RUM-event (front-end/user action); log-based (signal exists only in logs).
Q4	So stakeholders can slice the KPI the way the business thinks — by region, plan, tier, payment method — enabling far more questions to be answered from one metric.
Q5	Conversion rate = purchases ÷ visits × 100. Cart abandonment = (1 – purchases ÷ add-to-carts) × 100.
Q6	Banking: transaction success rate (successful ÷ total), payment processing time (latency of a payment). SaaS: active users (DAU/MAU), subscription conversion (trials converting to paid ÷ trials). (Others acceptable.)
Q7	Leading predicts (checkout latency, failed-transaction %); lagging reports after the fact (monthly churn, revenue). Observability favours leading KPIs because you can watch them change in real time and act before the lagging outcome moves.
Q8	Adding to cart is not buying — users abandon between the two; a rising add-to-cart with flat revenue would falsely signal health.

Module 08

#	Answer
Q9	session > view > (action, resource, error, long task).
Q10	The view event.
Q11	LCP (loading — largest element render), INP (interactivity — responsiveness), CLS (visual stability — unexpected shift), FCP (loading support — first content paint).
Q12	$LCP \leq 2.5$ s; $INP \leq 200$ ms; $CLS \leq 0.10$ (all "good").
Q13	INP replaced FID (March 2024); it is stricter because it reflects the worst interaction latency across the whole visit, not just the first input's delay.
Q14	The 75th percentile — averages hide the slower quarter of visits, understating real pain (same reasoning as latency percentiles on Day 02).
Q15	A privacy-conscious, video-like replay of a real user's DOM/session; best for understanding <i>what exactly happened</i> in a problematic session (e.g. rage-clicks, a confusing flow, the moment of an error).
Q16	By slicing data by device/browser/OS/country/connection/version/custom attributes to reveal which cohort is affected — e.g. "checkout error only on Safari/iOS in the EU."
Q17	LCP: optimise/preload hero image, CDN, remove render-blocking resources. INP: reduce main-thread long tasks / break up JS. CLS: reserve space for images/ads, don't insert content above existing content.

Module 09

#	Answer
Q18	Scheduled, simulated requests/interactions from chosen locations; its advantage is proactive, consistent, 24/7 coverage of critical paths independent of real traffic (catches issues before users do).
Q19	API test (single endpoint), multistep API test (chained workflow), browser test (end-to-end UI journey), mobile test (native app flows).
Q20	A pass/fail condition. Examples: status code = 200; response body contains a field; latency < 500 ms; SSL cert not expiring within N days.
Q21	Global = Datadog-managed locations worldwide (external availability/latency); private = run inside your network for internal or pre-prod endpoints.
Q22	It chains multiple requests and passes variables between them (e.g. auth token → order → verify), modelling a real transaction rather than pinging one endpoint.
Q23	RUM only: real-user diversity, field Core Web Vitals, session replay. Synthetics only: health of a path with no/low real traffic, pre-launch and off-hours, a consistent baseline/SLA.
Q24	Synthetics detect the outage proactively and alert; RUM quantifies real-user impact (how many, which devices/regions); APM shows the backend root cause. Detection → impact → cause.

Appendix B — KPI & Core Web Vitals cheat sheet

Business KPI formulas

KPI	Formula
Conversion rate	$\text{purchases} \div \text{visits} \times 100$
Cart abandonment rate	$(1 - \text{purchases} \div \text{add-to-carts}) \times 100$
Average order value	$\text{total revenue} \div \text{orders}$
Revenue per minute	$\Sigma \text{ order value per minute}$
Transaction success rate	$\text{successful} \div \text{total attempted} \times 100$
Failed transaction %	$\text{failed} \div \text{total} \times 100$
Stickiness (SaaS)	$\text{DAU} \div \text{MAU}$
Churn rate	$\text{customers lost} \div \text{customers at period start} \times 100$
Subscription conversion	$\text{trials converting to paid} \div \text{trials} \times 100$

Core Web Vitals thresholds (evaluate at p75)

Metric	Facet	Good	Needs improvement	Poor
LCP	Loading	$\leq 2.5 \text{ s}$	2.5–4.0 s	$> 4.0 \text{ s}$
INP	Interactivity	$\leq 200 \text{ ms}$	200–500 ms	$> 500 \text{ ms}$
CLS	Visual stability	≤ 0.10	0.10–0.25	> 0.25
FCP	Loading (support)	$\leq 1.8 \text{ s}$	1.8–3.0 s	$> 3.0 \text{ s}$

RUM vs Synthetics

Choose...	When you want...
RUM	Real-user experience, field Core Web Vitals, segmentation, session replay
Synthetics	Proactive 24/7 checks on critical paths, uptime/SLA, pre-launch & off-hours coverage

One-page memory hooks

Business obs: technical = means, business = end; correlate for attribution. **Instrument via:** custom / span-based / RUM / log-based. **Funnel KPIs:** conversion, cart abandonment, revenue/min. **Leading vs lagging:** watch leading (latency) to protect lagging (revenue). **RUM:** session > view > action/resource/error/long task; CWV live on views. **CWV:** LCP $\leq 2.5s$, INP $\leq 200ms$ (replaced FID), CLS ≤ 0.1 , FCP $\leq 1.8s$; judge at p75. **Synthetics:** API/multistep/browser/mobile; assertions; global vs private; proactive 24/7. **Together:** Synthetics detect, RUM shows impact, APM shows cause.

Appendix C — Sources & further reading

Claims about RUM, Core Web Vitals, and Synthetic Monitoring are grounded in the official Datadog and web.dev documentation below. Core Web Vitals thresholds follow Google's published values; confirm current defaults against the live docs.

- Datadog Docs — Real User Monitoring: docs.datadoghq.com/real_user_monitoring/
- Datadog Docs — Monitoring page performance & Core Web Vitals: docs.datadoghq.com/real_user_monitoring/browser/monitoring_page_performance/
- Datadog Docs — Session Replay: docs.datadoghq.com/real_user_monitoring/session_replay/
- Datadog Docs — Synthetic Monitoring: docs.datadoghq.com/synthetics/
- Datadog Docs — API & browser tests: docs.datadoghq.com/synthetics/api_tests/ · [/ browser_tests/](https://docs.datadoghq.com/synthetics/browser_tests/)
- Datadog Docs — Generate metrics (custom, span-based, log-based, RUM): docs.datadoghq.com/metrics/
- web.dev — Core Web Vitals & thresholds (LCP, INP, CLS, FCP); INP replacing FID (March 2024).

Day 03 study guide compiled August 2026. Diagrams are original, produced for this guide. UI screenshots are intentionally left as labelled placeholders to capture live from your own Datadog account. Module 09 (Synthetic Monitoring) is included to complete the "3 modules / real and simulated journey" framing of the Day 03 outline.